CSP-203: Software Tools & Technologies Lab



Chapter – 5

Version Control Systems: Git

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

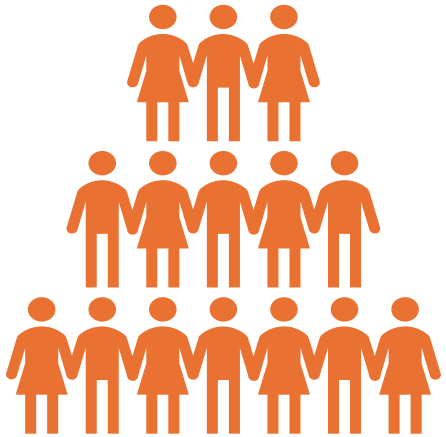
Version Control – For Individual



Does any of the following sound familiar?

- Your code was working great! Then you made a few changes, and now everything is broken, and you saved over the previous version?
- You accidentally delete a critical file and can't get it back.
- Your computer is broken or was stolen, and now all of your files are gone!
- While writing a paper for one of your classes, you save each version as final_paper.doc, final_paper2.doc, final_paper_actually_this_time.doc.
- There has to be a better way to manage versions...

Version Control – For Teams



Does any of the following sound familiar?

- My partner and I are paired up for a project for one of our CS classes. We usually pair program together in the labs, but sometimes we have to work remotely.
  - Who keeps the most up-to-date version of the project?
  - How do we share changes with each other?
  - What if I want to compare the changes my partner made?

- How do we keep backups of important files? Who stores them on their computer?













TOP VERSION CONTROL SYSTEMS

Version Control

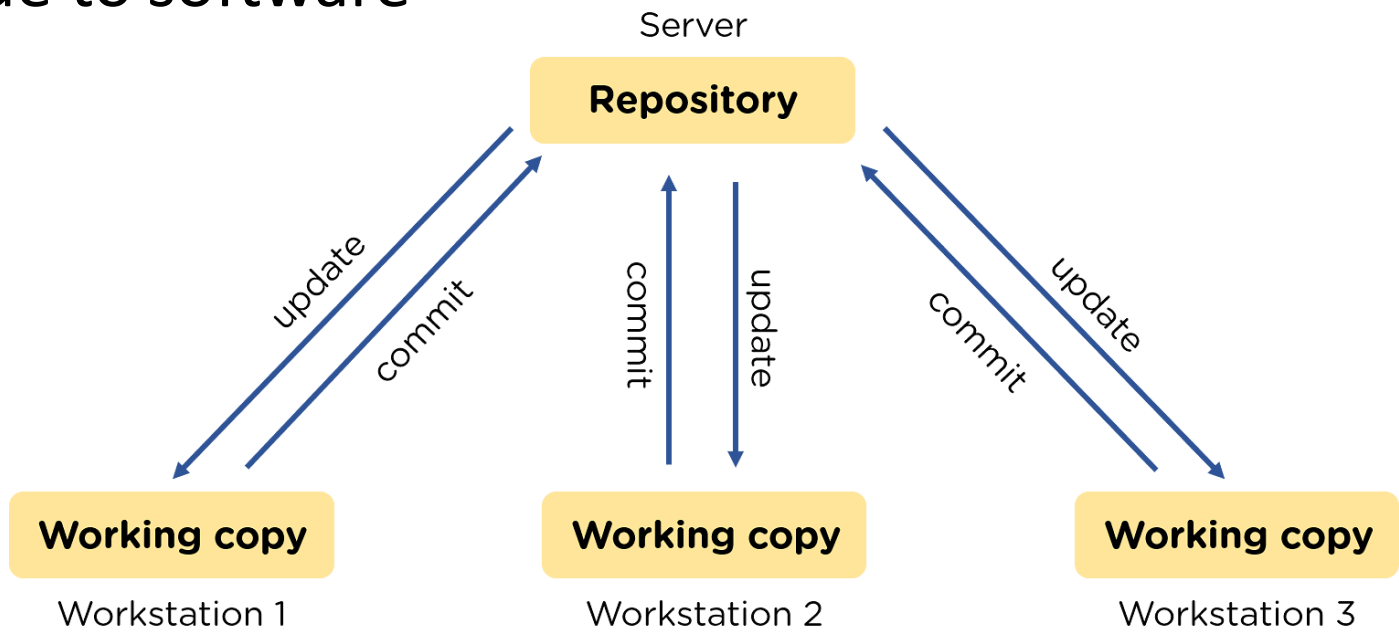
Version control is the practice of tracking and managing changes to software code.

Benefits of version control systems

1. A complete long-term change history of every file.
2. Branching and merging : Helps team members work concurrently.
3. Traceability : Tracing changes made to software







Git and GitHub

Overview

Git is a popular version control system. It was created by Linus Torvalds in 2005, and has been maintained by Junio Hamano since then.

What does Git do?

- Manage projects with ***Repositories***
- Clone a project to work on ***a local copy***
- Control and track changes with ***Staging and Committing***

GitHub is a web-based hosting service for git repositories

One can use git without Github,but you cannot use GitHub without Git

Git

Setup

Windows

Download the package from https://git-scm.com/download/win

Linux

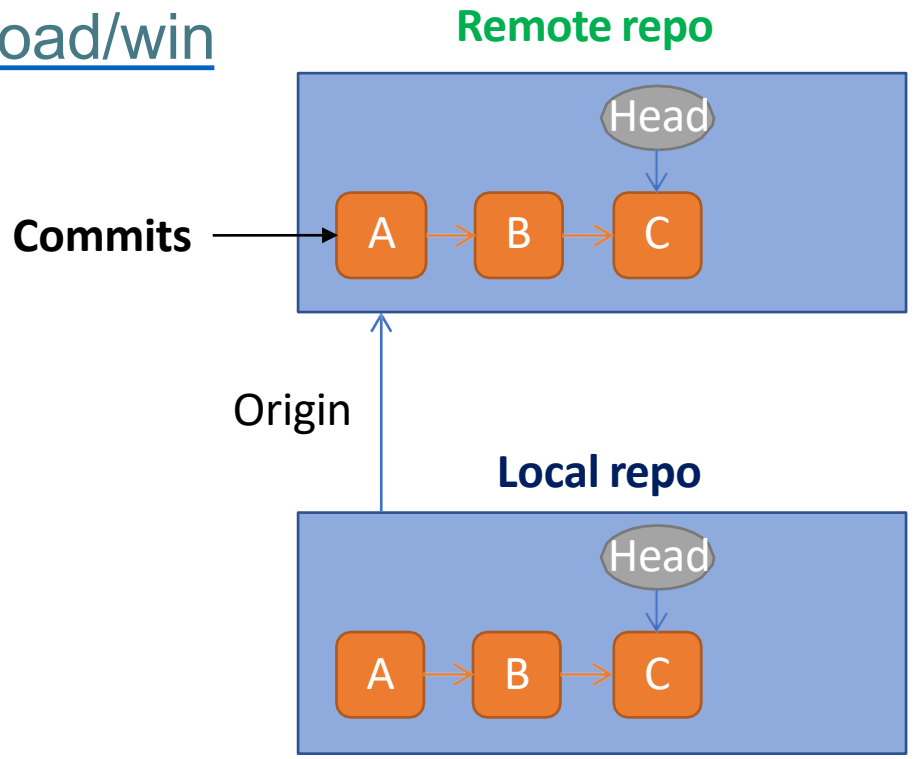
Use the command *sudo apt install git-all*

Use git –version to check the version

‘Repository’ – term used to represent storage

There are 2 types of Repositories

- Local – On our machine
- Remote – GitHub.com



Short History of Git:

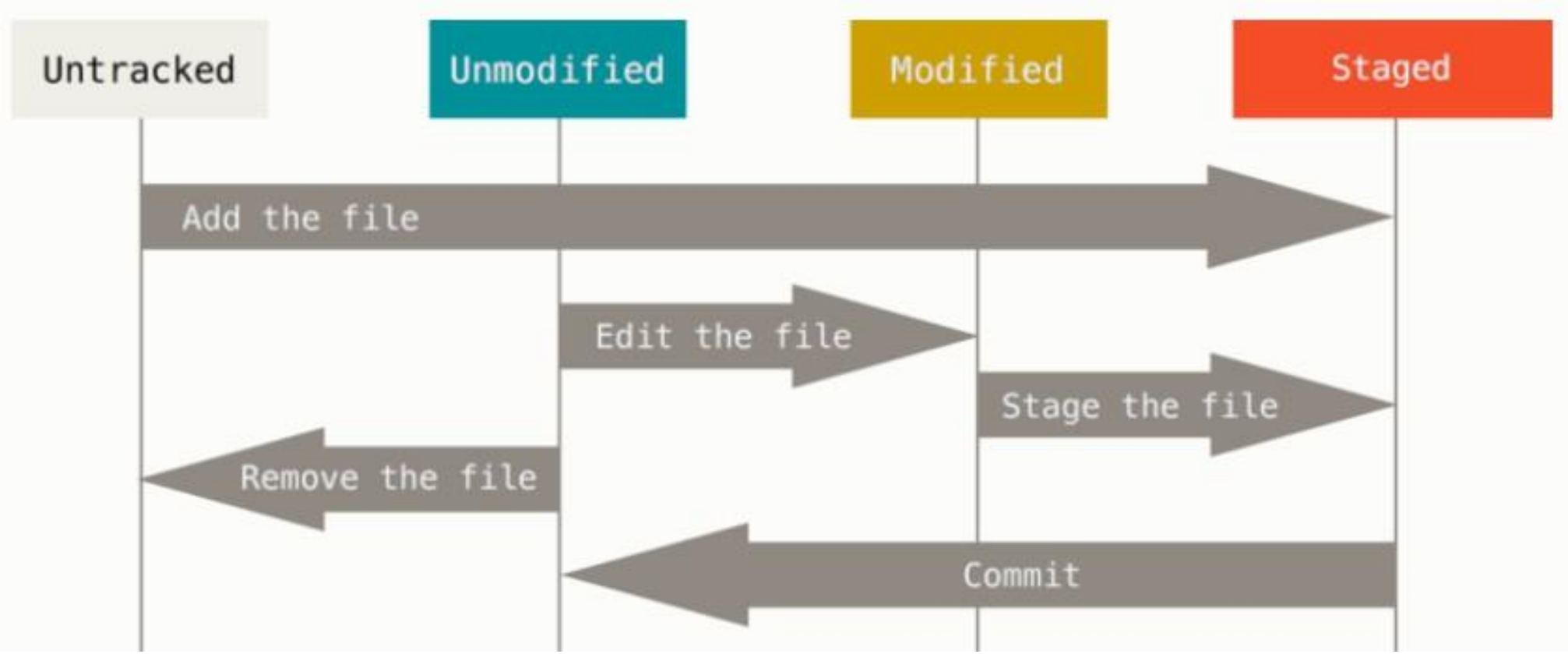
- The Linux kernel is an open source software project of fairly large scope.
- During the early years of the Linux kernel maintenance (1991–2002), changes to the software were passed around as patches and archived files.
- In 2002, the Linux kernel project began using a proprietary DVCS called BitKeeper.
- In 2005, the relationship between the community that developed the Linux kernel and the commercial company that developed BitKeeper broke down, and the tool's free-of-charge status was revoked.
- This prompted the Linux development community (and in particular Linus Torvalds, the creator of Linux) to develop their own tool based on some of the lessons they learned while using BitKeeper.

Introduction to Git, A Newer Version Control System:

- Git has Three States:
  - Git has three main states that your files can reside in: modified, staged, and committed:
    - Modified means that you have changed the file but have not committed it to your database yet.
    - Staged means that you have marked a modified file in its current version to go into your next commit snapshot.
    - Committed means that the data is safely stored in your local database.
  - The basic Git workflow goes something like this:
    - You modify files in your working tree.
    - You selectively stage just those changes you want to be part of your next commit, which adds only those changes to the staging area.
    - You do a commit, which takes the files as they are in the staging area and stores that snapshot permanently to your Git directory.

Git (Recording Changes to the Repositories)

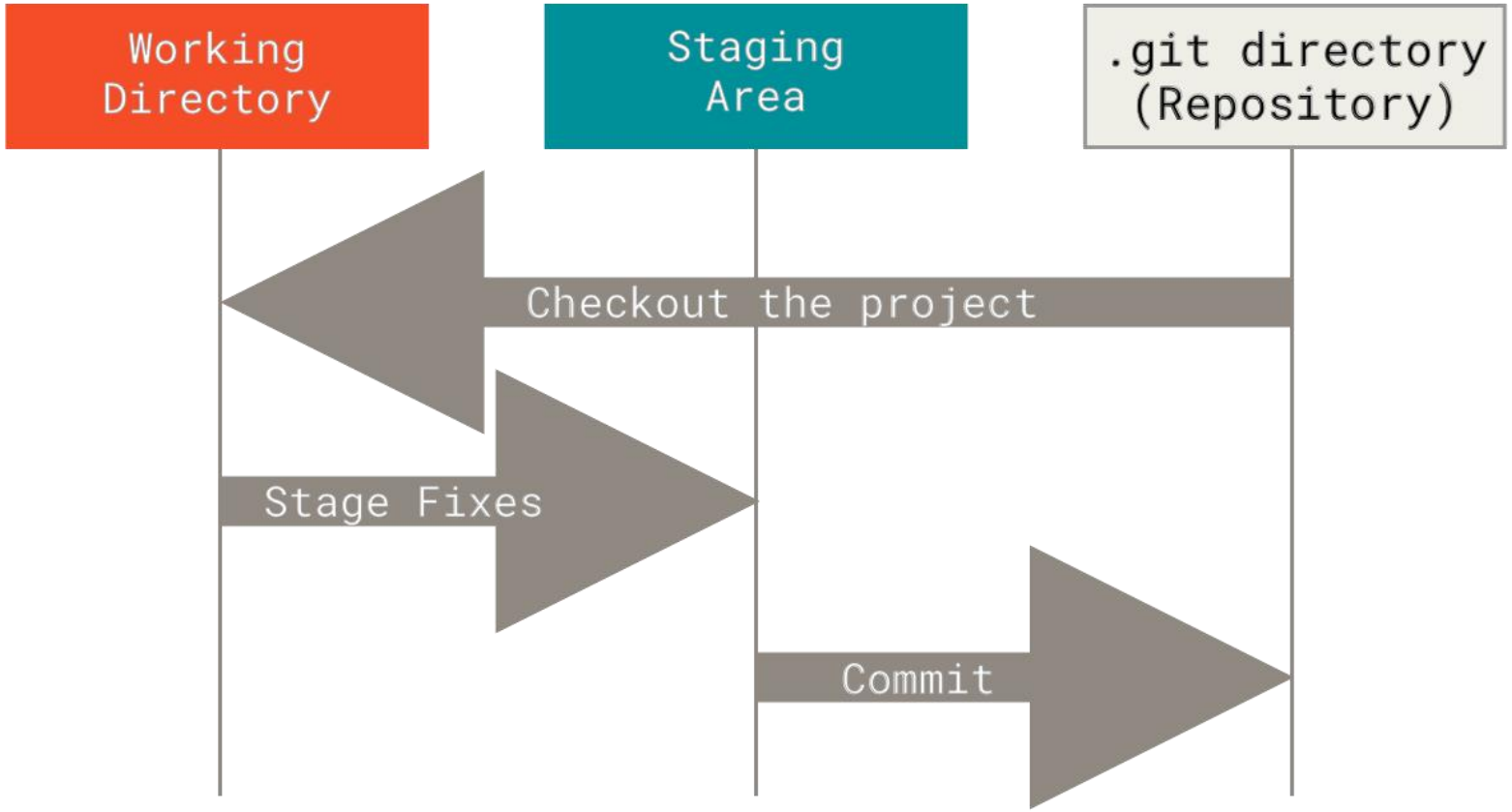
Lifecycle of a File



Introduction to Git, A Newer Version Control System:

➢ Git has Three States:

- If a particular version of a file is in the Git directory, it's considered committed.
- If it has been modified and was added to the staging area, it is staged. I
- If it was changed since it was checked out but has not been staged, it is modified.



Starting with Git:

- Installing git:
  - For Linux: sudo apt install git-all; git --version
  - For MacOs: If you want a more up to date version, you can also install it via a binary installer. A macOS Git installer is maintained and available for download at the Git website, at https://git-scm.com/download/mac .
  - For Windows: Installing on Windows
  - There are also a few ways to install Git on Windows. The most official build is available for download on the Git website. Just go to https://git-scm.com/download/win and the download will start automatically.

First Time Git Setup:

- ➢ You can view all of your settings and where they are coming from using: $ git config --list --show-origin
- ➢ Your Identity:
  - The first thing you should do when you install Git is to set your user name and email address. $ git config --global user.name "John Doe”

    $ git config --global user.email johndoe@example.com

First Time Git Setup:

- ➢ Checking Settings:
  - If you want to check your configuration settings, you can use the git config --list command to list all the settings Git can find at that point: $ git config --list

```
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto
...
```

  - You may see keys more than once, because Git reads the same key from different files. In this case, Git uses the last value for each unique key it sees.
  - You can also check what Git thinks a specific key's value is by typing git config <key>:

    $ git config user.name

    John Doe

Getting Help:

- If you ever need help while using Git, there are three equivalent ways to get the comprehensive manual page (manpage) help for any of the Git commands:
  - $ git help <verb>
  - $ git <verb> --help
  - $ man git-<verb>
- For example, you can get the manpage help for the git config command by running this:
  - $ git help config
- In addition, if you don't need the full-blown manpage help, but just need a quick refresher on the available options for a Git command, you can ask for the more concise "help" output with the -h option, as in:
  - $ git add -h

Initializing a Repository in an Existing Directory:

- Initializing a Repository in an Existing Directory:
  - If you have a project directory that is currently not under version control and you want to start controlling it with Git, you first need to go to that project's directory.
  - $ cd /home/user/my_project $ git init
  - This creates a new subdirectory named .git that contains all of your necessary repository files — a Git repository skeleton.

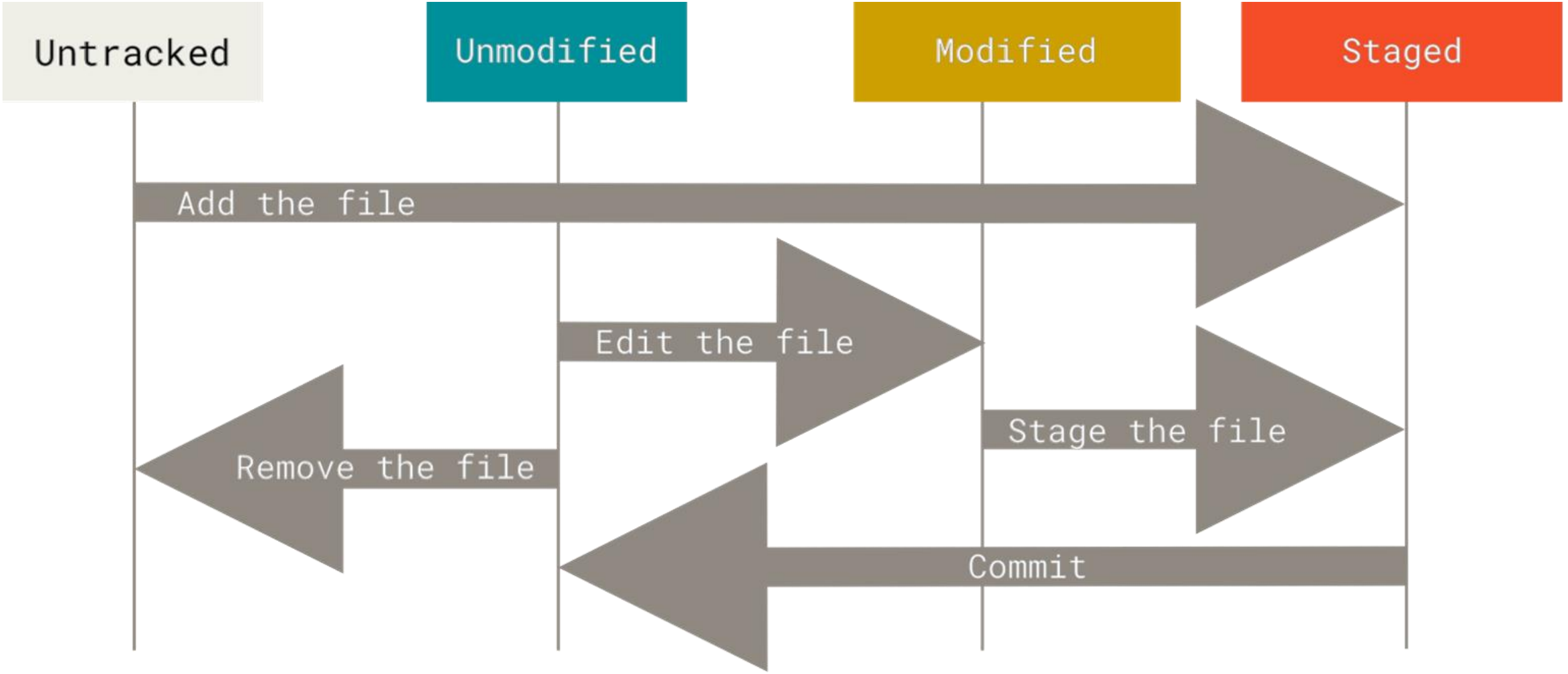
Cloning an Existing Repository:

- You clone a repository with git clone <url>. For example, if you want to clone the Git linkable library called libgit2, you can do so like this:
  - $ git clone https://github.com/libgit2/libgit2
- If you want to clone the repository into a directory named something other than libgit2, you can specify the new directory name as an additional argument:
  - $ git clone https://github.com/libgit2/libgit2 mylibgit
  - That command does the same thing as the previous one, but the target directory is called mylibgit.
- Git has a number of different transfer protocols you can use. The previous example uses the https:// protocol, but you may also see git:// or user@server:path/to/repo.git, which uses the SSH transfer protocol.

Recording Changes to the Repositories:

- ➢ Each file in your working directory can be in one of two states: tracked or untracked.
- ➢ Tracked files are files that were in the last snapshot, as well as any newly staged files; they can be unmodified, modified, or staged. In short, tracked files are files that Git knows.
- ➢ Untracked files are everything else — any files in your working directory that were not in your last snapshot and are not in your staging area.
- ➢ When you first clone a repository, all of your files will be tracked and unmodified because Git just checked them out and you haven't edited anything.
- ➢ As you edit files, Git sees them as modified, because you've changed them since your last commit.
- ➢ As you work, you selectively stage these modified files and then commit all those staged changes, and the cycle repeats.

Recording Changes to the Repositories:



Checking the Status of Your Files:

- The main tool you use to determine which files are in which state is the git status command. If you run this command directly after a clone, you should see something like this: $ git status
- Let's say you add a new file to your project, a simple README file. If the file didn't exist before, and you run git status, you see your untracked file like so:
  - $ echo 'My Project' > README
  - $ git status
- Your new README file is untracked, because it's under the "Untracked files" heading in your status output. Untracked basically means that Git sees a file you didn't have in the previous snapshot (commit), and which hasn't yet been staged
- Git won't start including it in your commit snapshots until you explicitly tell it to do so.

Tracking New Files:

- In order to begin tracking a new file, you use the command git add. To begin tracking the README file, you can run this: $ git add README

Staging Modified Files:

- Let's change a file that was already tracked. If you change a previously tracked file called CONTRIBUTING.md and then run your git status command again.
- The CONTRIBUTING.md file appears under a section named "Changes not staged for commit" — which means that a file that is tracked has been modified in the working directory but not yet staged. To stage it, you run the git add command.
- git add is a multipurpose command — you use it to begin tracking new files, to stage files, and to do other things like marking merge-conflicted files as resolved
- You can stage all changes (new, modified, deleted files) at once: $git add --all or git add .

Viewing Your Staged and Unstaged Changes:

- What have you changed but not yet staged?
- What have you staged that you are about to commit?
  - $ git status answers those questions very generally by listing the file names
  - $ git diff shows you the exact lines added and removed — the patch, as it were.
- If you want to see what you've staged that will go into your next commit, you can use git diff --staged.
- It's important to note that git diff by itself doesn't show all changes made since your last commit — only changes that are still unstaged.
- If you've staged all of your changes, git diff will give you no output.

Committing Your Changes:

- The simplest way to commit is to type git commit: $ git commit
- Alternatively, you can type your commit message inline with the commit command by specifying it after a -m flag, like this: $ git commit -m "Story 182: fix benchmarks for speed"

Skipping the Staging Area:

- The staging area is sometimes a bit more complex than you need in your workflow. If you want to skip the staging area, adding the -a option to the git commit command makes Git automatically stage every file that is already tracked before doing the commit.

Fetching and Pulling from Your Remotes:

- As you just saw, to get data from your remote projects, you can run: $ git fetch <remote>
- git fetch command only downloads the data to your local repository — it doesn't automatically merge it with any of your work or modify what you're currently working on.
- You can use the git pull command to automatically fetch and then merge that remote branch into your current branch.
- The git clone command automatically sets up your local master branch to track the remote master branch (or whatever the default branch is called) on the server you cloned from.

Pushing to Your Remotes:

- When you have your project at a point that you want to share, you have to push it upstream. The command for this is simple: $ git push <remote> <branch>.

Git Branching and Merging:

- $ git branch [branch-name]
- $ git checkout master
- $ git merge developer

Git

Creating Repositories

- Cloning a Repository from a remote server to the local machine

```
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl$ git clone https://gitlab.eurecom.fr/oai/cn5g/oai-cn5g-fed
Cloning into 'oai-cn5g-fed'...
warning: redirecting to https://gitlab.eurecom.fr/oai/cn5g/oai-cn5g-fed.git/
remote: Enumerating objects: 7316, done.
remote: Counting objects: 100% (181/181), done.
remote: Compressing objects: 100% (180/180), done.
remote: Total 7316 (delta 81), reused 0 (delta 0), pack-reused 7135
Receiving objects: 100% (7316/7316), 43.29 MiB | 110.00 KiB/s, done.
Resolving deltas: 100% (5284/5284), done.
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl$ ls
demo  oai-cn5g-fed
```

Git clone is used to clone remote repositories from remote servers such as GitHub

```
$ git clone https://gitlab.eurecom.fr/cn5g/oai-cn5g-fed.git
```

Protocol **Server** **Path to repository on server**

Git

Creating Repositories

2. **git init** for initializing a local repository

```
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl$ mkdir demo
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl$ cd demo/
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ git init
Initialized empty Git repository in /home/snsrl4/Documents/ssl/demo/.git/
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ ls -a
.  ..  .git
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$
```

git init converts a directory to Git local repo

Git

Adding files to Repo

git add

```
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ touch Readme.md
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Readme.md

nothing added to commit but untracked files present (use "git add" to track)
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ git add Readme.md
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ git status
```

git status shows the status of untracked files

git add is used to add files to the repository

When changes are made to a added file, we need to add the file again using the command **git add**

Git

Checking the difference between the staged files and the current files

git diff

```
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ touch demo.txt
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ git add demo.txt
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ echo "Welcome to SSL" > demo.txt
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ git diff
diff --git a/demo.txt b/demo.txt
index e69de29..c480115 100644
--- a/demo.txt
+++ b/demo.txt
@@ -0,0 +1 @@
+Welcome to SSL
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$
```

git diff shows the files that are modified from the last add.

Git

git commit for saving changes in local repository

git commit

```
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$ git commit -m "Added demo.txt"
[master (root-commit) f180d64] Added demo.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 demo.txt
snsrl4@snsrl4-B560M-DS3H-V2:~/Documents/ssl/demo$
```

‘commit’ changes i.e. save all the changes (adding a new file in this example) in the local repository

GitHub

- Git clone a repo
- Add a new file
- Stage and commit the file to the repo
- Push the staged file
- Observe the changes in github

Thank you!

Email: anand@iitbhilai.ac.in